which results in a `params` hash like

```
{'person' => {'birth_date(1i)' => '2008', 'birth_date(2i)' => '11',
'birth_date(3i)' => '22'}}
```

When this is passed to `Person.new` (or `update`), Active Record spots that these parameters should all be used to construct the `birth_date` attribute and uses the suffixed information to determine in which order it should pass these parameters to functions such as `Date.civil`.

4.3 Common Options

Both families of helpers use the same core set of functions to generate the individual select tags and so both accept largely the same options. In particular, by default Rails will generate year options 5 years either side of the current year. If this is not an appropriate range, the `:start_year` and `:end_year` options override this. For an exhaustive list of the available options, refer to the API documentation.

As a rule of thumb you should be using `date_select` when working with model objects and `select_date` in other cases, such as a search form which filters results by date.

In many cases the built-in date pickers are clumsy as they do not aid the user in working out the relationship between the date and the day of the week.

4.4 Individual Components

Occasionally you need to display just a single date component such as a year or a month. Rails provides a series of helpers for this, one for each component `select_year`, `select_month`, `select_day`, `select_hour`, `select_minute`, `select_second`. These helpers are fairly straightforward. By default they will generate an input field named after the time component (for example, "year" for `select_year`, "month" for `select_month` etc.) although this can be overridden with the `:field_name` option. The `:prefix` option works in the same way that it does for `select_date` and `select_time` and has the same default value.

The first parameter specifies which value should be selected and can either be an instance of a `Date`,`Time` or `DateTime`, in which case the relevant component will be extracted, or a numerical value. For example:

```
<%= select_year(2009) %>
<%= select_year(Time.now) %>
```

will produce the same output if the current year is 2009 and the value chosen by the user can be retrieved by `params[:date][:year]`.

5 Uploading Files

A common task is uploading some sort of file, whether it's a picture of a person or a CSV file containing data to process. The most important thing to remember with file uploads is that the rendered form's encoding **MUST** be set to "multipart/form-data". If you use `form_for`, this is done automatically. If you use `form_tag`, you must set it yourself, as per the following example.

The following two forms both upload a file.

```
<%= form_tag({action: :upload}, multipart: true) do %>
  <%= file_field_tag 'picture' %>
<% end %>

<%= form_for @person do |f| %>
  <%= f.file_field :picture %>
<% end %>
```

Rails provides the usual pair of helpers: the barebones `file_field_tag` and the model oriented `file_field`. The only difference with other helpers is that you cannot set a default value for file